

DCA0305

Lesson 12 · November 9

morsinaldo.medeiros.075@ufrn.edu.br
ivanovitch.silva@ufrn.br

AI Agents III · Multi-agent

One problem, three architectures, measured

Where we are

10

Agents I

The loop and tools, by hand

→

11

MCP

Tools as a shared standard

→

12

Agents III

Today · many agents, frameworks, measuring

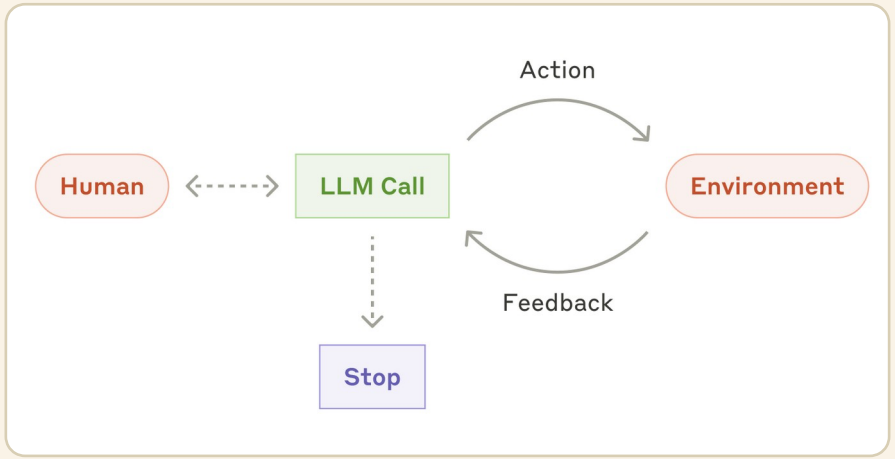
Why more than one agent

Who checks the text? · When to split · The price · Why they fail



So far: one agent, one loop

Lesson 10 built the loop; lesson 11 moved the tools behind MCP



The agent loop · Anthropic, "Building effective agents" (2024)

- **One model** qwen in Ollama decides every step
- **One loop** Reason → act → observe, until it answers
- **One server** search_cars · car_details · compare_cars over MCP

Today: what changes when more than one agent shares the work?

Who checks the text?

The numbers come from the tools; the sentences come from the model

TOOL RESULT · LESSON 11

```
compare_cars(["c081", "c082"])

"best": {
  "lowest_km":       ["c082"],
  "most_economical": ["c081"],
  "cheapest":        ["c081"]
}
```

→

*"I would pick **c081**, it has **fewer kilometres**."*

Wrong: c082 has fewer km

A second agent could read both and object. That is the first reason to split.

Good reasons to split



Context that bloats

One context window fills with tool results. A subagent works in a fresh window and returns only a short report.



Parallel work

Independent pieces, such as searching ten sources, run at the same time in separate agents.

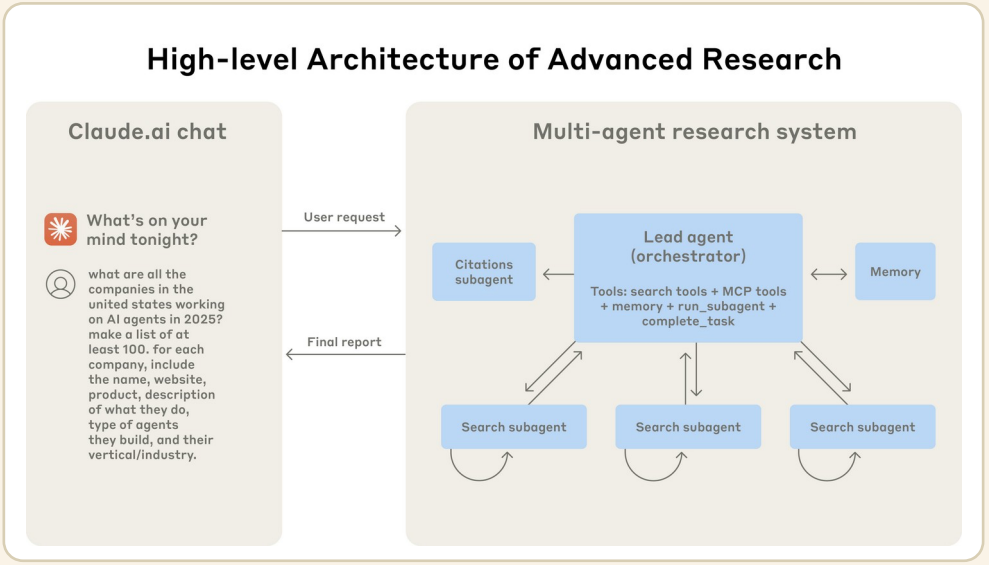


Independent check

An agent that did not write the answer reviews it against the data, like a second pair of eyes.

The price of splitting

Anthropic's multi-agent research system · June 2025



Lead agent + parallel search subagents · anthropic.com/engineering/multi-agent-research-system

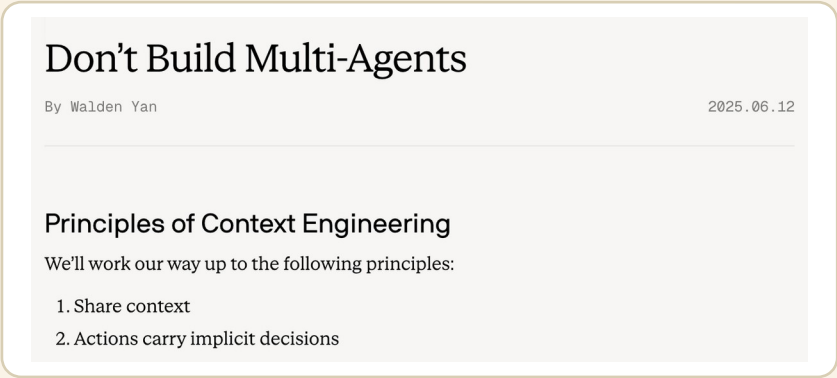
+90.2% Better than a single agent on their internal research eval

~4× Tokens of a single agent, compared with a chat

~15× Tokens of the multi-agent system, compared with a chat

Two posts

June 2025: one says do not split, the other splits and wins



Don't Build Multi-Agents

By Walden Yan 2025.06.12

Principles of Context Engineering

We'll work our way up to the following principles:

1. Share context
2. Actions carry implicit decisions

Cognition · Walden Yan · June 12, 2025

Don't split Agents that do not share context make conflicting decisions



How we built our multi-agent research system

Claude now has Research capabilities that allow it to search across the web, Google Workspace, and any integrations to accomplish complex tasks.

The journey of this multi-agent system from prototype to production taught us critical lessons about system architecture, tool design, and prompt engineering. A multi-agent system consists of multiple agents (LLMs autonomously using tools in a loop) working together. Our Research feature involves an agent that plans a research process based on user queries, and then uses tools to create parallel agents that search for information simultaneously. Systems with multiple agents introduce new challenges in agent coordination, evaluation, and reliability.

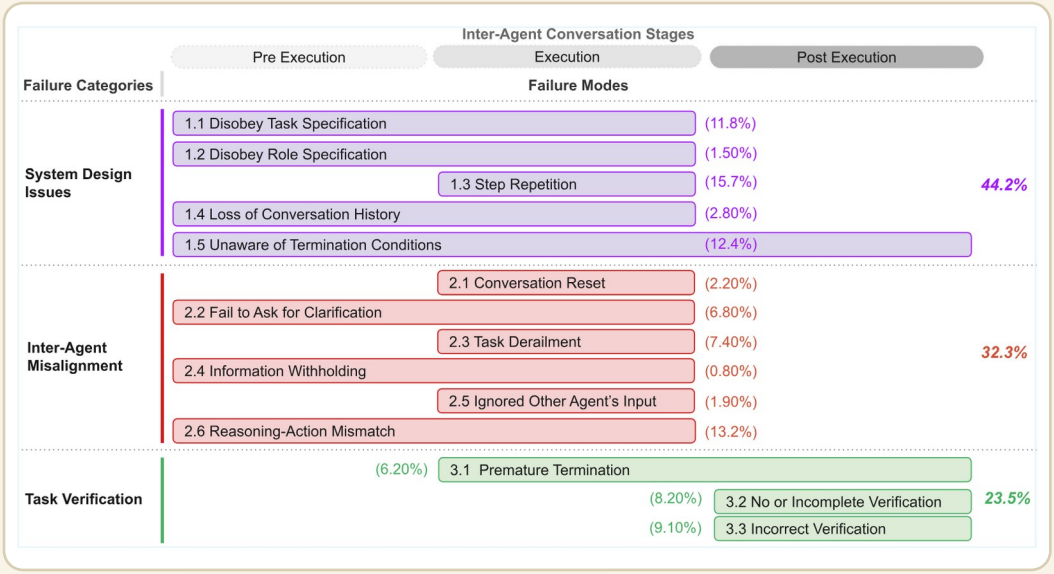
Published Jun 13, 2025
Team Engineering
Topics Agents, Claude apps, Tool use
See more...

Anthropic · June 13, 2025

Split Independent searches in parallel: +90.2% on research questions

Why multi-agent systems fail

MAST: 14 failure modes in 1,600+ traces from 7 frameworks · Cemri et al. (2025)



- **System design · 44%**
- **Inter-agent misalignment · 32%**
- **Task verification · 24%**

Figure 1 of "Why Do Multi-Agent LLM Systems Fail?" · arxiv.org/abs/2503.13657

76% of the failures: the system design, or agents misaligned with each other.

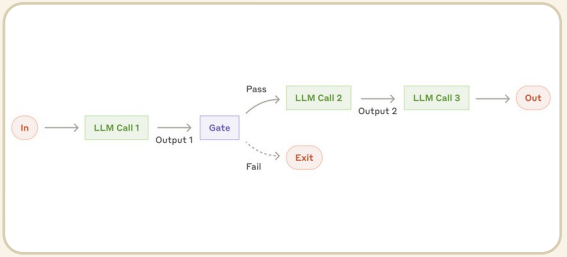
Patterns

Pipeline · Routing · Parallel · Supervisor · Verifier

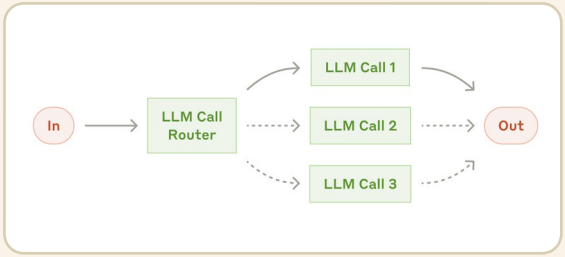
II

Five shapes, one question

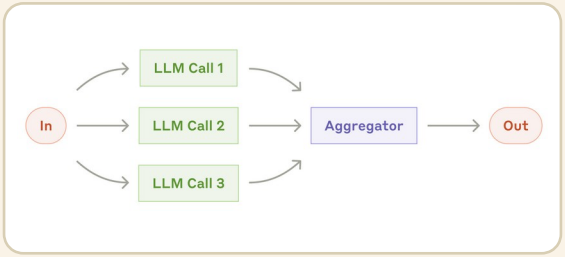
Who decides what happens next: your code or a model?



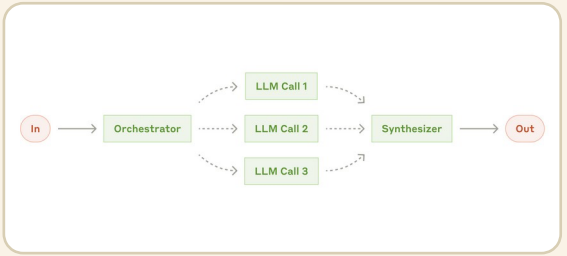
Pipeline Code decides



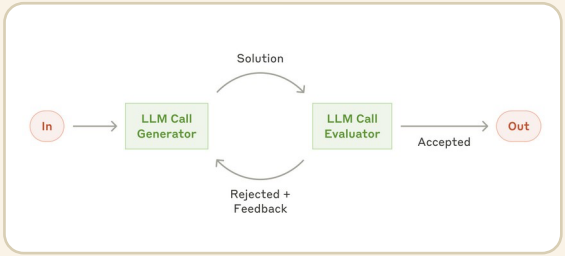
Routing A model picks one path



Parallel Code splits and joins



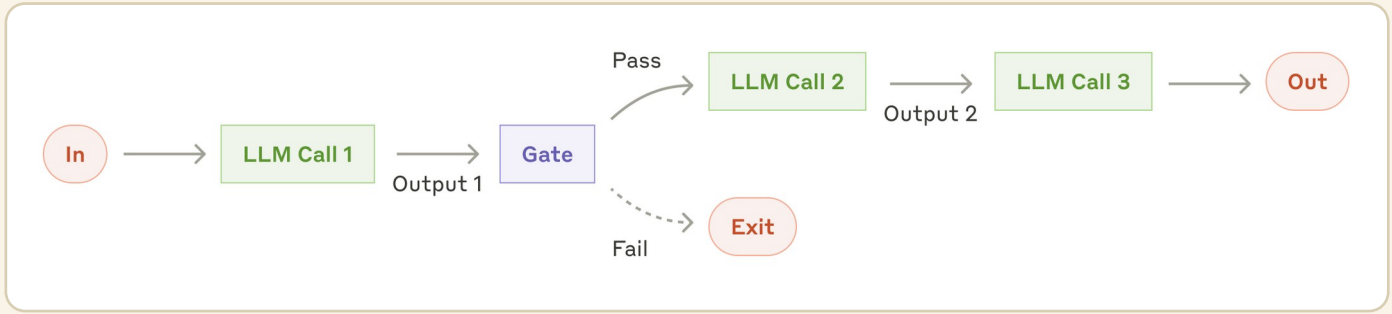
Orchestrator A model splits the work



Generator + verifier A model approves or sends back

Diagrams from Anthropic, "Building effective agents" (December 2024)

Pipeline: a fixed sequence

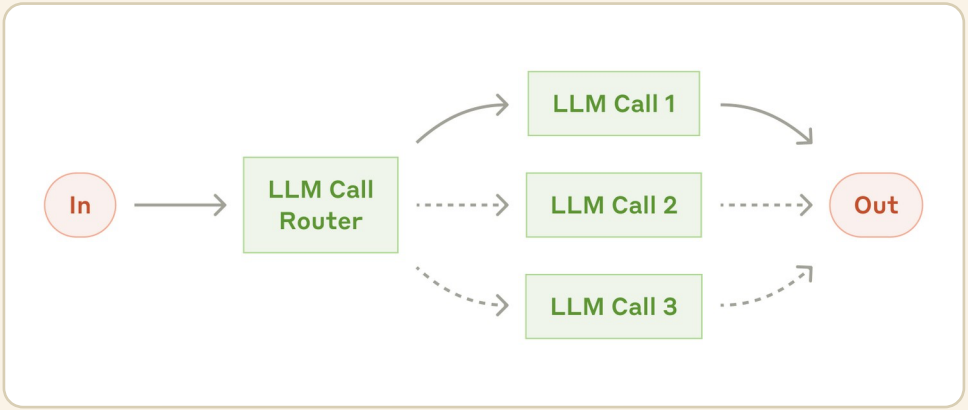


Search → **Compare** → **Write**

Who decides the next step? Your code

Cheap and predictable. Breaks when the request does not fit the sequence.

Routing: one model picks the path



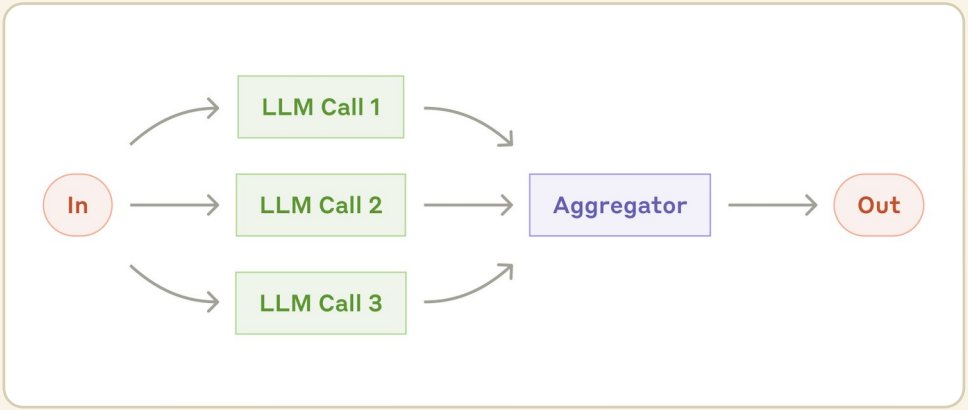
Routing · Anthropic (2024)

- **Router** Classifies the request, once
- **Handlers** One per kind: search, details, compare
- **Out of scope** A polite answer, no tools called

Who decides the next step? A model, once, at the start

Cheaper than a loop when the kinds of request are few and easy to tell apart.

Parallel: split, run, join



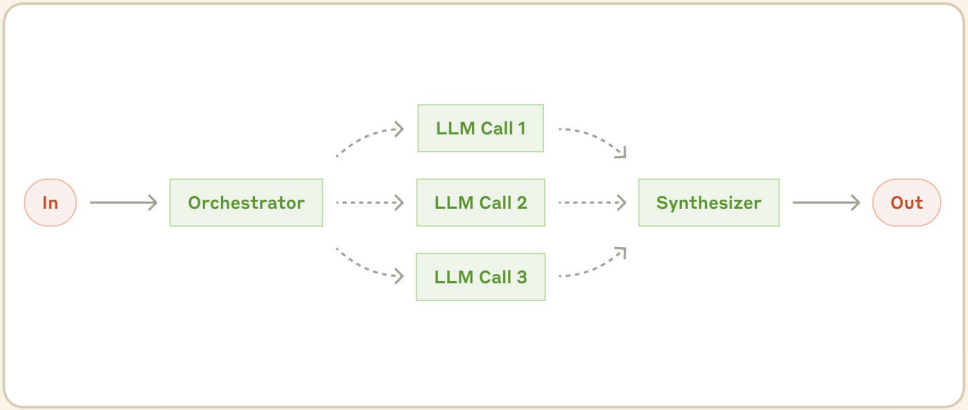
Parallelization · Anthropic (2024)

- **Sectioning** Independent parts at once: c081 and c082
- **Voting** Same task several times; keep the majority
- **Aggregator** Your code joins the results

Who decides the next step? Your code: split, run, join

Faster when the parts are independent. It is the shape of Anthropic's research system (slide 7).

Supervisor + specialists



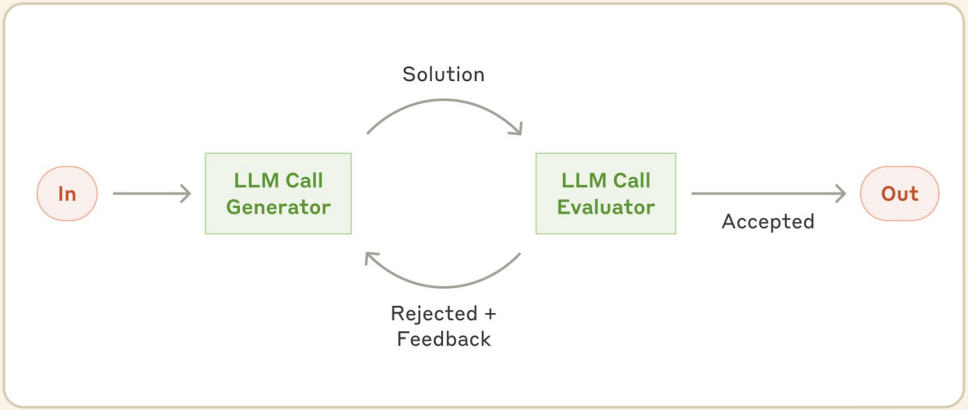
Orchestrator-workers · Anthropic (2024)

- **Supervisor** Reads the request, picks who acts
- **Specialists** Each with its own tools
- **Writer** Turns the reports into the answer

Who decides the next step? The supervisor, after each report

The specialist sees only the task it was given. Whatever the supervisor leaves out is lost.

Generator + verifier



Evaluator-optimizer · Anthropic (2024)

- **Generator** Our lesson 11 agent
- **Verifier** Reads answer + tool results
- **Rejected** Back to the generator, with the reason

Who decides the next step? The verifier: approve or send back

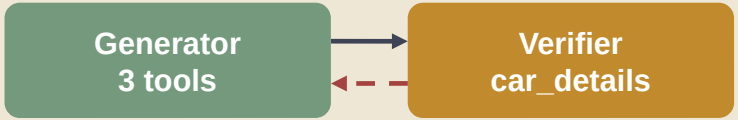
Works when there is a clear criterion to check against, here the stock data.

Who decides the next step?

Pattern	Who decides	Good for	Watch out
Pipeline	Your code	Fixed, known steps	Requests off the script
Routing	A model, once	Distinct kinds of request	A wrong route is final
Parallel	Your code	Independent parts, votes	Merging conflicting parts
Supervisor	The supervisor	Mixed requests, many tools	Context lost in the task
Generator + verifier	The verifier	Answers with a clear check	A verifier that also errs

Our two candidates, on the car stock

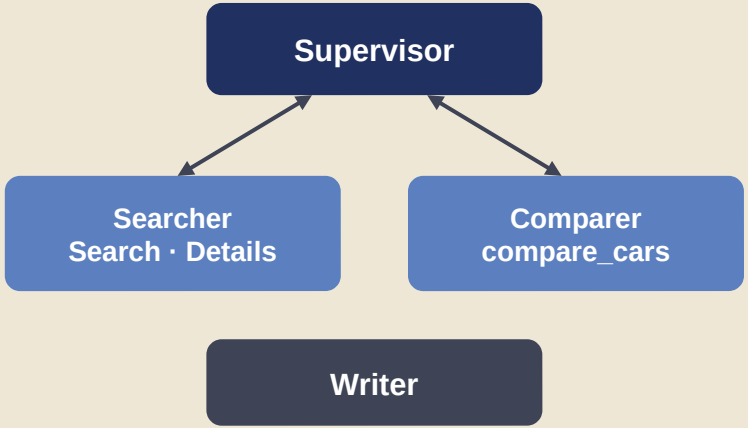
Generator + verifier



Objection needs evidence

Code looks up each car_id in the answer; the model only judges.

Supervisor + specialists



Same server.py, same model (qwen3:8b), same questions. Only the architecture changes.

Frameworks

LangGraph · CrewAI · One server for both · Traces

III

LangGraph: the loop becomes a graph

LESSON12.IPYNB · PART 1

```python
g = StateGraph(Estado)
g.add_node("modelo", no_modelo)
g.add_node("ferramentas", ToolNode(tools))

g.add_edge(START, "modelo")
g.add_conditional_edges("modelo",
    tools_condition,
    {"tools": "ferramentas", END: END})
g.add_edge("ferramentas", "modelo")

agente = g.compile()
```

- **State** The messages list
- **Node** One step: the model or the tools
- **Conditional edge** Tool call? Loop. Text? End.

LangGraph overview

Copy page

LangGraph is a low-level orchestration framework and runtime for long-running, stateful agents. You define workflows as graphs of shared state, nodes, and edges, and can mix deterministic steps with LLM-driven steps in the same graph.

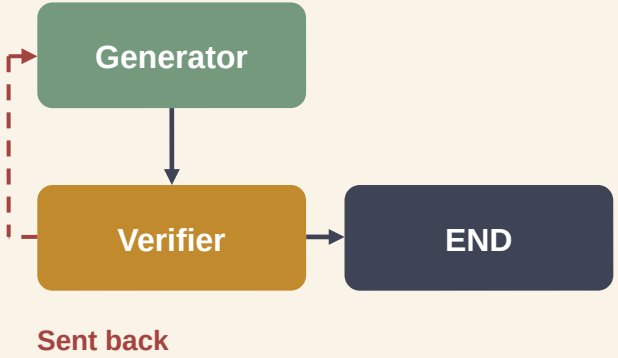
LangGraph focuses on orchestration capabilities such as durable execution, streaming, and human-in-the-loop. It does not prescribe prompts or agent architecture.

Adding a verifier is adding a node

PART 2

```
g.add_node("verificador", no_verificador)

g.add_conditional_edges("gerador",
    depois_do_gerador,     # tool call?
    ["ferramentas", "verificador"])
g.add_conditional_edges("verificador",
    depois_do_verificador, # approved?
    ["gerador", END])
```



LangGraph beyond the loop

Three features you will want in your project



Checkpointer

Saves the state at every step: memory between turns, resume after a crash

```
saver = InMemorySaver()
app = g.compile(
    checkpointer=saver)
cfg = {"configurable":
    {"thread_id": "ana"}}
```



Human in the loop

Pauses before a risky step and waits for a person to approve

```
ok = interrupt("Reserve c082?")

# later, with the answer:
app.invoke(
    Command(resume=True), cfg)
```



Streaming

Shows each node as it finishes, instead of waiting for the end

```
async for p in app.astream(x,
    stream_mode="updates"):
    print(p)
```

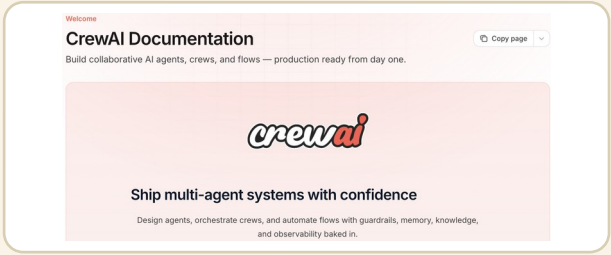
CrewAI: describe the team, not the flow

PART 5

```
vendedor = Agent(
    role="Used-car assistant in Natal",
    goal="Answer with real cars ...",
    backstory="You never invent a car.",
    tools=ferramentas, llm=llm_crew)
verificador = Agent(role="Fact checker", ...)

crew = Crew(agents=[vendedor, verificador],
    tasks=[responder, conferir],
    process=Process.sequential)
```

- **Agent** Role, goal, backstory, tools
- **Task** What to do and who does it
- **Crew** Sequential or hierarchical



One MCP server, two frameworks

The lesson 11 server.py, unchanged

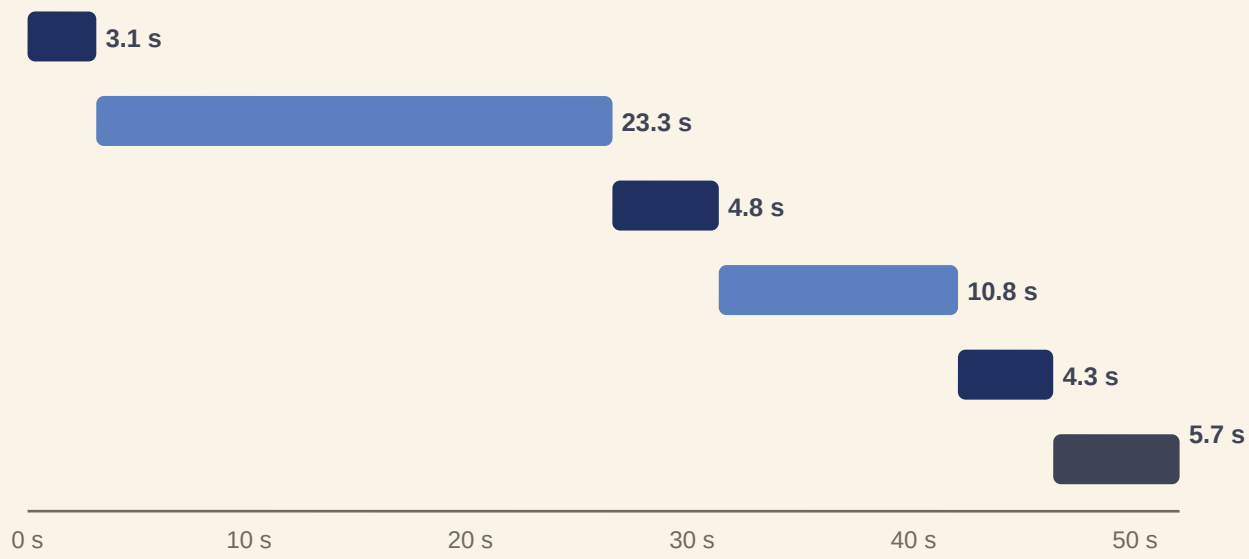


Two library versions that cannot share one Python. They do not need to: the server is another program, and they only exchange messages.

Seeing inside: a trace

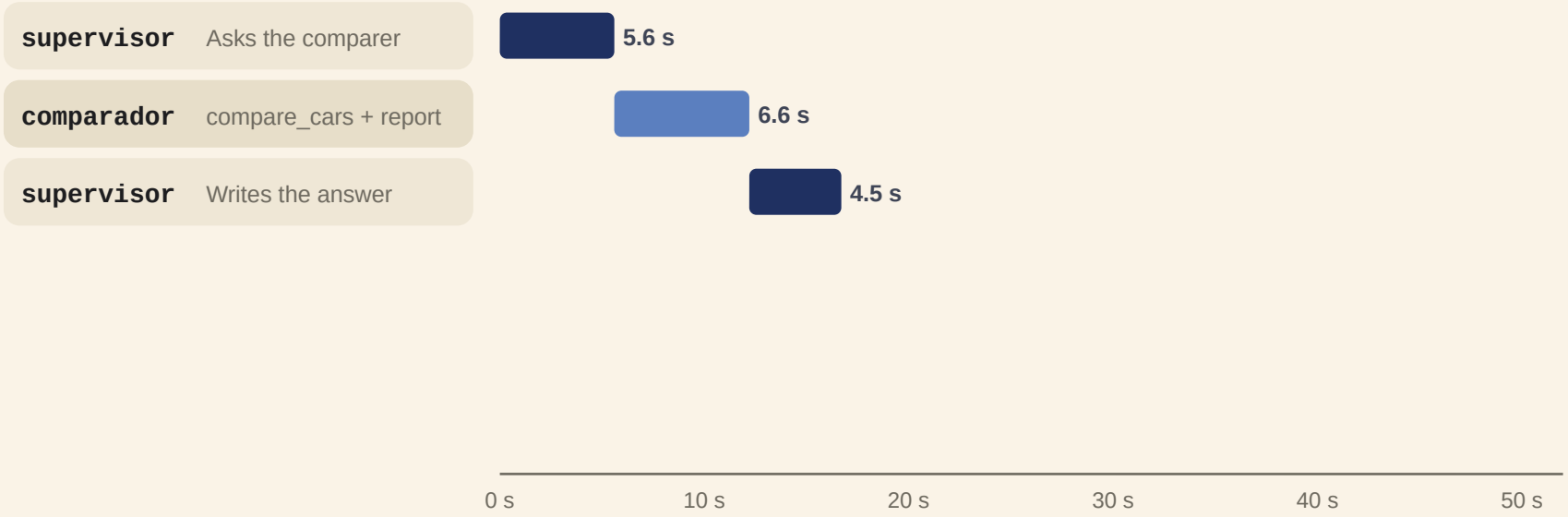
LangGraph's supervisor answering "c081 or c082?", step by step · Measured

supervisor	Routes to the searcher
buscador	car_details ×2 + report
supervisor	Routes to the comparer
comparador	compare_cars + report
supervisor	Decides to answer
redator	Writes the answer



The same trace, in CrewAI

CrewAI's supervisor answering "c081 or c082?", step by step · Measured



Which one for your project?

	No framework	LangGraph	CrewAI
You describe	A while-loop	Nodes, edges, state	Roles, goals, tasks
The loop is	Yours, in plain sight	In plain sight	Inside the Agent
Control	Total	High	Medium
Speed to a demo	Slow	Medium	Fast
We used it in	Lesson 10	Parts 1–4	Part 5

Others follow the same ideas: Google ADK, smolagents, OpenAI Agents SDK, Claude Agent SDK (lesson 16).

Measure

IV

Trajectories · Two graders · The numbers · Judging the judge

One question, three trajectories

Check the answer, the path and the cost · "c081 or c082?" · Measured

Single agent



8.6 s · 2 calls · Wrong: says c082

Generator + verifier



45.8 s · 5 calls · Right, after one send-back

Supervisor



52.0 s · 8 calls · Right

The last two got it right, for 5 to 6 times the time of the single agent.

Eight questions, answers from the data

Search	A family SUV up to 80k	`search_cars`
Search	Newest hatch up to 50k	`search_cars`
Details	City and colour of c109	`car_details`
Details	Consumption and trunk of c003	`car_details`
Compare	c081 or c082: more economical?	`compare_cars`
Compare	c109 or c061: larger trunk?	`compare_cars`
Trap	Compare the two newest SUVs	`compare_cars`
Missing	Colour of c999?	`Must say "not found"`

Two graders, and where they disagree

Automatic checks

Regex and rules: right car_id? Car within the filter? Right tool called?

LLM judge

Reads question, reference and answer; says correct or not, with a reason.

Same answer, two verdicts

"O carro c082 é o mais econômico, com um consumo de 10,4 km/l, em comparação ao c081, que tem um consumo de 10,9 km/l."

Auto ✓ **Judge ✗**

Rules only catch what you foresaw. The judge catches meaning, but it can err too, so it gets the reference.

The numbers

Eight questions · qwen3:8b · Apple M4, local Ollama · Same checks and judge

	LLM judge		Model calls		Seconds	
	LangGraph	CrewAI	LangGraph	CrewAI	LangGraph	CrewAI
Single agent	**7/8**	**8/8**	**17**	**18**	**101**	**114**
Generator + verifier	**8/8**	**8/8**	**27**	**30**	**259**	**202**
Supervisor	**7/8**	**8/8**	**52**	**46**	**367**	**352**

Verifier: +1 right answer for 1.6× the calls. Supervisor: ~3× the calls in both frameworks, no gain.

Question by question

LangGraph · LLM judge verdict · Model calls · Seconds

	Single agent	Generator + verifier	Supervisor
1 · Family SUV up to 80k	✓ 2 calls · 22 s	✓ 3 calls · 54 s	✓ 8 calls · 103 s
2 · Newest hatch up to 50k	✓ 2 calls · 9 s	✓ 3 calls · 20 s	✓ 8 calls · 38 s
3 · City and colour of c109	✓ 2 calls · 4 s	✓ 3 calls · 21 s	✗ 5 calls · 14 s
4 · Consumption and trunk of c003	✓ 2 calls · 5 s	✓ 3 calls · 18 s	✓ 5 calls · 15 s
5 · c081 or c082: more economical?	✗ **2 calls · 8 s**	✓ 5 calls · 43 s	✓ 8 calls · 50 s
6 · c109 or c061: larger trunk?	✓ 2 calls · 9 s	✓ 3 calls · 20 s	✓ 5 calls · 43 s
7 · Compare the two newest SUVs	✓ 3 calls · 37 s	✓ 4 calls · 64 s	✓ 8 calls · 87 s
8 · Colour of c999?	✓ 2 calls · 7 s	✓ 3 calls · 19 s	✓ **5 calls · 17 s**

Reading the table

- **The verifier earned its cost** — It fixed question 5 (c081 vs c082) for 1.6× the calls and 2.6× the time
- **The supervisor did not, twice** — ~3× the calls in both frameworks; in LangGraph it also said c109 is black
- **The framework writes the prompt** — CrewAI's single agent got q5 right: same model, other text
- **Graders err too** — The rule failed "não foi encontrado"; the judge passed "c082 has more km" (it has less)

The judge needs checking too

Zheng et al. (2023), MT-Bench · Yao et al. (2024), τ-bench

Position bias

Comparing two answers, it favours one position, often the first it reads

Verbosity bias

It prefers the longer answer, even when it says nothing more

Self-enhancement

It favours answers written by its own model. Ours: qwen3 judging qwen3

pass^k

Run each question k times; it counts only if all k pass. GPT-4o: pass^8 under 25% (retail)

What we did: one answer at a time, a reference in every prompt, and we read every disagreement.

Start with one; split when numbers ask



Anthropic, "Building agents: when to fold subagents into your main agent loop" (Oct 2026)

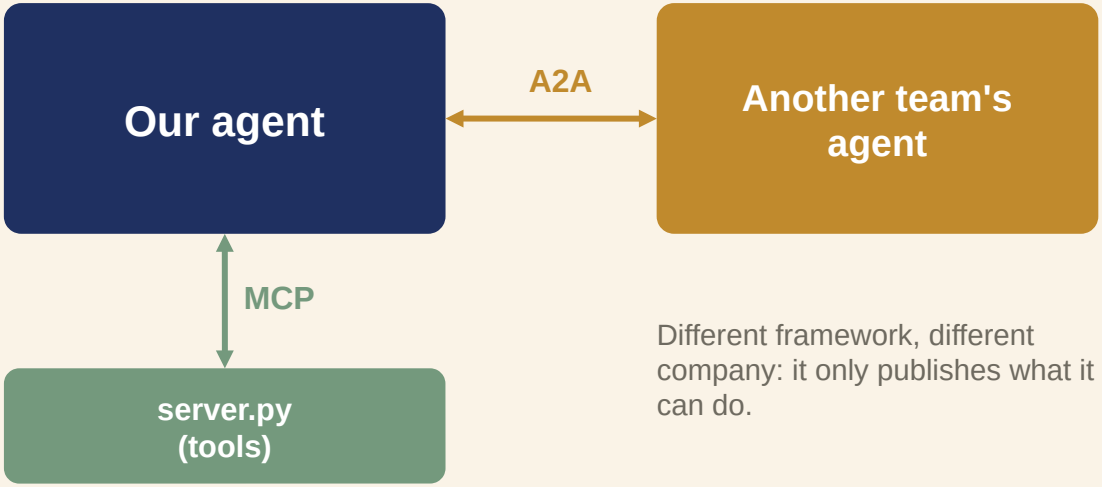
- **Build the evals first** Ten golden examples are enough to start
- **Split by context** Not by problem type, not by team
- **Change one thing at a time** And compare task by task, not only the total
- **Keep a subagent only if…** The single-loop version scores worse

Agents talking to agents

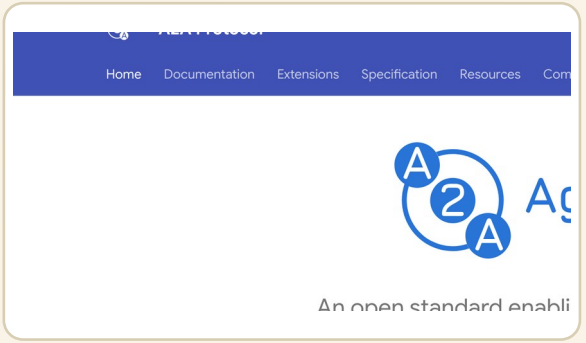
V

MCP and A2A · Agent Card · Task lifecycle

MCP connects tools; A2A connects agents



Different framework, different company: it only publishes what it can do.



a2a-protocol.org

"Not competitors, highly complementary." MCP: agent to tool. A2A: agent to agent.

A2A in four concepts

Agent Card

A JSON document: who the agent is, its skills, endpoint and authentication

Task

A unit of work with an ID and a lifecycle, from submitted to completed

Message

One turn between client and agent, made of parts: text, file or data

Artifact

The concrete output of a task, which can be streamed

Apr 2025 Google announces A2A

Jun 2025 Donated to the Linux Foundation

2026 Version 1.0; joins the Agentic AI Foundation

Over HTTP, with JSON-RPC 2.0 messages, the same format as MCP; streaming with Server-Sent Events.

An Agent Card for our car agent

Served at /.well-known/agent-card.json · A2A 1.0 format, illustrative

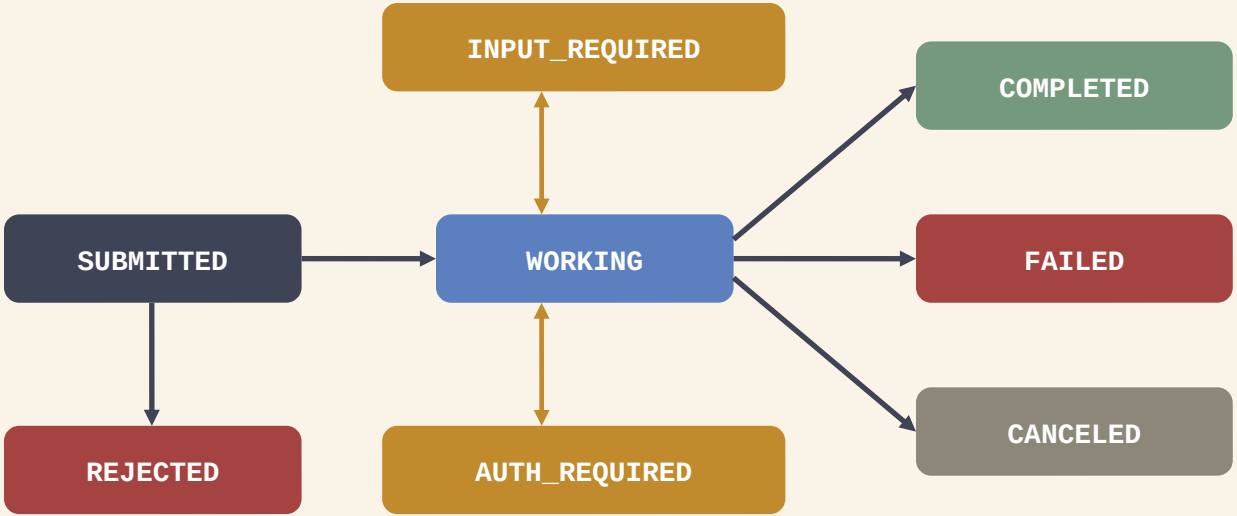
```
{
  "name": "Natal Used-Car Agent",
  "description": "Recommends used cars from the Natal stock",
  "supportedInterfaces": [{
    "url": "https://cars.example.com/a2a/v1",
    "protocolBinding": "JSONRPC", "protocolVersion": "1.0"}],
  "version": "1.0.0",
  "capabilities": {"streaming": true},
  "defaultInputModes": ["text/plain"],
  "defaultOutputModes": ["text/plain", "application/json"],
  "skills": [{"id": "recommend-car",
    "name": "Recommend a car",
    "examples": ["A family SUV up to 80k"]}]
}
```

- **Who** Name, description, provider
- **Where** URL and binding: JSON-RPC, gRPC or HTTP+JSON
- **What** Skills, with examples of requests
- **How** Capabilities and security schemes

No code, no prompt, no tools: only what it can do.

A task's lifecycle

States are TASK_STATE_* in A2A 1.0



Example

A dealer's agent asks ours: "SUV up to 80k".

`WORKING → INPUT_REQUIRED`
"Automatic or manual?"

`WORKING → COMPLETED`
With an artifact: the cars.

Amber states wait for the client. **Green, red and grey ones end the task. The client polls, streams (SSE) or gets a push.**

References & further reading

Anthropic (2024) **Building effective agents** `anthropic.com/engineering/building-effective-agents`

Anthropic (2025) **How we built our multi-agent research system** `anthropic.com/engineering/multi-agent-research-system`

Anthropic (2026) **Building agents: when to fold subagents into your main agent loop** `claude.com/resources/articles`

Lanham (2025) **AI Agents in Action, chapters 4 and 11** `Manning`

LangGraph · CrewAI **Official documentation** `docs.langchain.com/oss/python/langgraph · docs.crewai.com`

A2A Protocol **Specification 1.0 and key concepts** `a2a-protocol.org`

Cognition (2025) **Don't build multi-agents** `cognition.com/blog/dont-build-multi-agents`

Cemri et al. (2025) **Why do multi-agent LLM systems fail?** `arxiv.org/abs/2503.13657`

Zheng et al. (2023) **Judging LLM-as-a-judge with MT-Bench and Chatbot Arena** `arxiv.org/abs/2306.05685`

Yao et al. (2024) **τ-bench: tool-agent-user interaction** `arxiv.org/abs/2406.12045`